

Esercizi sull'analisi di complessità

1. Determinare la funzione di costo $T(n)$ del seguente algoritmo nel caso migliore e nel caso peggiore, essendo n la lunghezza dell'array A .

CONTIENE_DUPLICATI(A)	
1	$i=1$
2	while $i \leq A.length$
3	$j=i+1$
4	while $j \leq A.length$
5	if $A[i] == A[j]$
6	return TRUE
7	$j=j+1$
8	$i=i+1$
9	return FALSE

2. Determinare la funzione di costo $T(n)$ del seguente algoritmo nel caso migliore e nel caso peggiore, essendo n il parametro della funzione.

ALGO1(n)	
1	$s=0$
2	$a=n$
3	while $a > 0$
4	$s=s+a$
5	$a=a/2$
6	return s

3. Determinare la funzione di costo $T(m,n)$ del seguente algoritmo nel caso migliore e nel caso peggiore, essendo m ed n rispettivamente il numero di righe ed il numero di colonne della matrice M .

ALGO2(M)	
1	$m=A.rows$
2	$n=A.columns$
3	$s=0$
4	$i=1$
5	while $i \leq m$
6	$j=1$
7	while $j \leq n$
8	$s=s+A[i][j]$
9	$j=j+1$
10	$i=i*2$
11	return s

4. Determinare l'andamento asintotico della funzione di costo $T(n)$ nel caso peggiore e nel caso migliore per il seguente algoritmo, essendo n il numero di elementi dell'array A .

ALGO3(A)	
1	$i=1$
2	$s=0$
2	while $i \leq A.length$
3	$s=s+A[i]$
4	$i=i*2$
5	return s

5. Determinare l'andamento asintotico della funzione di costo $T(n)$ nel caso peggiore e nel caso migliore per il seguente algoritmo, essendo n il numero di elementi dell'array A .

ALGO4(A)	
1	for $i=1$ to $A.length$
2	for $j=A.length$ downto i
3	if $A[i]==A[j]$
4	return TRUE
5	return FALSE

6. Determinare l'andamento asintotico della funzione di costo $T(n)$ nel caso peggiore e nel caso migliore per il seguente algoritmo, essendo n il numero di elementi dell'array A .

ALGO5(A)	
1	$a=0$
2	for $i=2$ to $A.length-1$
3	for $j=i-1$ to $i+1$
4	$a==a+A[j]$
5	return a

7. Determinare l'andamento asintotico della funzione di costo $T(m,n)$ nel caso peggiore e nel caso migliore per il seguente algoritmo, essendo m ed n rispettivamente il numero di righe e di colonne della matrice M .

ALGO6(M)	
1	$c=0$
2	for $i=1$ to $M.rows$
3	$j=1$
4	while $j \leq M.columns$
5	$c=c+M[i][j]$
6	if $M[i][j] \bmod 2 == 0$
7	$j=j*2$
8	else
9	$j=j+1$
10	return c

8. Determinare l'andamento asintotico della funzione di costo $T(n)$ nel caso peggiore e nel caso migliore per il seguente algoritmo, essendo n il numero di elementi dell'array A .

ALGO8(A)	
1	$k=0$
2	for $i=1$ to $A.length$
3	$k=k+i$
4	$a=0$
5	for $i=1$ to k
6	$j=i \bmod A.length$
7	$a=a+A[j]$
8	return a

9. Determinare l'andamento asintotico della funzione di costo $T(n)$ nel caso peggiore e nel caso migliore per il seguente algoritmo, essendo n il numero di elementi dell'array A .

ALGO7(A)	
1	$a=0$
2	for $i=1$ to $A.length$
3	$j=1$
4	$k=1$
5	$continua=TRUE$
6	while $continua$
7	$a=A[i][j]$
8	if $j==A.length$ and $k<A.length$
9	$k=k+1$
10	$j=1$
11	elseif $j==A.length$ and $k==A.length$
12	$continua=FALSE$
13	else
14	$j=j+1$
15	return a

Soluzioni

1. Il caso peggiore si ha quando l'array A non contiene duplicati. In questo caso, l'istruzione alla riga 6 non viene mai eseguita e i due cicli vengono ripetuti il massimo numero di volte. Il caso migliore si ha invece quando il primo e il secondo elemento sono uguali. In questo caso la prima volta che viene eseguito il controllo alla riga 5, esso risulta vero e l'algoritmo termina.

CONTIENE_DUPLICATI(A)		Caso Peggiore	Caso Migliore
1	$i=1$	1	1
2	while $i \leq A.length$	$n + 1$	1
3	$j=i+1$	n	1
4	while $j \leq A.length$	$n - i + 1 \quad \forall i = 1, \dots, n,$	1
5	if $A[i] == A[j]$	$n - i \quad \forall i = 1, \dots, n,$	1
6	return TRUE	0	1
7	$j=j+1$	$n - i, \quad \forall i = 1, \dots, n,$	0
8	$i=i+1$	n	0
9	return FALSE	1	0

$$\begin{aligned}
 T_{worse}(n) &= 1 + n + 1 + n + \sum_{i=1}^n (n - i + 1 + n - i + n - i) + n + 1 = \\
 &= 3n + \sum_{i=1}^n (3n - 3i + 1) = 3n + 3 + 3n \sum_{i=1}^n 1 - 3 \sum_{i=1}^n i + \sum_{i=1}^n 1 = \\
 &= 3n + 3 + 3n^2 - 3 \frac{n(n+1)}{2} + n = \frac{3}{2}n^2 + \frac{5}{2}n + 3 \\
 T_{best}(n) &= 6
 \end{aligned}$$

2. In questo caso non esiste un caso peggiore e un caso migliore.

ALGO1(n)		
1	$s=0$	1
2	$a=n$	1
3	while $a>0$	$h + 1$
4	$s=s+a$	h
5	$a=a/2$	h
6	return s	1

Abbiamo la seguente espressione per $T(n)$ in cui h indica il numero di volte che si ripete il ciclo esterno.

$$T(n) = 3h + 4$$

Per valutare il valore di h assumiamo per semplicità che n sia una potenza di due (in questo modo ogni divisione intera per 2 non produrrà resto). Dopo una iterazione del ciclo abbiamo che $a = \frac{n}{2}$; dopo due iterazioni abbiamo $a = \frac{n}{4}$; dopo tre iterazioni $a = \frac{n}{8}$. In definitiva, dopo i iterazioni abbiamo $a = \frac{n}{2^i}$. Il ciclo terminerà dopo h iterazioni con h pari al minimo intero che rende vera la disequazione $\frac{n}{2^h} < 1$, da cui $n < 2^h$. Prendendo il logaritmo in base due di entrambi i membri abbiamo $\log_2 n < h$. Poiché h è il minimo intero che soddisfa l'equazione precedente e poiché n è una potenza di due abbiamo $h = \log_2 n + 1$.

Se h non è una potenza di due, dopo una iterazione abbiamo che $a = \left\lfloor \frac{a}{2} \right\rfloor$; dopo due iterazioni $a = \left\lfloor \frac{\left\lfloor \frac{n}{2} \right\rfloor}{2} \right\rfloor = \left\lfloor \frac{n}{4} \right\rfloor$. In definitiva dopo i iterazioni abbiamo $a = \left\lfloor \frac{n}{2^i} \right\rfloor$. Ragionando come nel caso precedente abbiamo che h sarà il minimo intero che soddisfa $\left\lfloor \frac{n}{2^h} \right\rfloor < 1$. Quest'ultima sarà soddisfatta solo se $\frac{n}{2^h} < 1$ da cui otteniamo, come nel caso precedente, $\log_2 n < h$. In questo caso n non è una potenza di due e quindi il minimo intero che soddisfa la precedente è $h = \lfloor \log_2 n \rfloor + 1$. In definitiva abbiamo $T(n) = 3\lfloor \log_2 n \rfloor + 7$.

3. In questo caso non esiste un caso peggiore e un caso migliore.

ALGO2(M)		
1	$m=A.rows$	1
2	$n=A.columns$	1
3	$s=0$	1
4	$i=1$	1
5	while $i \leq m$	$h + 1$
6	$j=1$	h
7	while $j \leq n$	$n + 1$ per ogni iterazione del ciclo esterno (quindi $h(n + 1)$ volte)
8	$s=s+A[i][j]$	n per ogni iterazione del ciclo esterno (quindi $h \cdot n$ volte)
9	$j=j+1$	n per ogni iterazione del ciclo esterno (quindi $h \cdot n$ volte)
10	$i=i*2$	h volte
11	return s	1

Abbiamo la seguente espressione per $T(m, n)$ in cui h indica il numero di volte che si ripete il ciclo esterno.

$$T(m, n) = 4 + h + 1 + h + h(n + 1) + 2h \cdot n + h + 1 = 3h \cdot n + 4h + 6$$

Per valutare il valore di h osserviamo che dopo una iterazione del ciclo abbiamo che $a = 2$; dopo due iterazioni abbiamo $a = 4$; dopo tre iterazioni $a = 8$. In definitiva, dopo i iterazioni abbiamo $a = 2^i$. Il ciclo terminerà dopo h iterazioni con h pari al minimo intero che rende vera la disequazione $2^h > m$. Prendendo il logaritmo in base due di entrambi i membri abbiamo $h > \log_2 m$. Poiché h è il minimo intero che soddisfa l'equazione precedente abbiamo $h = \lfloor \log_2 m \rfloor + 1$. In definitiva otteniamo:

$$T(m, n) = 3n \cdot \lfloor \log_2 m \rfloor + 3n + 4 \lfloor \log_2 m \rfloor + 4 + 6 = 3n \cdot \lfloor \log_2 m \rfloor + 3n + 4 \lfloor \log_2 m \rfloor + 10$$

4. In questo caso non esiste un caso peggiore e un caso migliore.

ALGO3(A)	
1	$i=1$
2	$s=0$
2	while $i \leq A.length$
3	$s=s+A[i]$
4	$i=i*2$
5	return s

Per determinare la complessità asintotica del metodo possiamo osservare che il corpo del ciclo while ha costo costante e quindi la complessità del ciclo while è data dal numero di volte h che si ripete. Le altre istruzioni (righe 1,2 e 5) hanno complessivamente un costo costante. Per valutare la complessità quindi

dobbiamo valutare quanto vale h . La variabile i viene inizializzata ad 1 e ad ogni iterazione raddoppia; dopo k iterazioni quindi vale 2^k . Ne segue che h sarà il minimo intero tale che $2^h > n$. Ragionando come nell'esercizio precedente è facile vedere che $h = \Theta(\lg n)$. Ne segue che $T(n) = \Theta(\lg n)$.

5. Il caso peggiore si ha quando gli elementi dell'array A sono tutti diversi; il caso migliore si ha quando il primo e l'ultimo elemento sono uguali.

ALGO4(A)	
1	for $i=1$ to $A.length$
2	for $j=A.length$ downto i
3	if $A[i]==A[j]$
4	return TRUE
5	return FALSE

Per determinare la complessità asintotica di caso peggiore osserviamo che l'istruzione alla riga 3 è l'istruzione dominante e quindi ci basta determinare quante volte essa viene eseguita. Il ciclo **for** più esterno viene ripetuto n volte (per i che va da 1 a n). Per ogni valore di i il ciclo più interno viene eseguito $n - i + 1$ volte. In definitiva il numero di volte che l'istruzione alla riga 3 viene eseguita è data da $\sum_{i=1}^n (n - i + 1)$. È facile vedere che $\sum_{i=1}^n (n - i + 1) = \Theta(n^2)$. La complessità di caso migliore è banalmente $\Theta(1)$ in quanto, nel caso migliore, l'istruzione dominante viene eseguita una sola volta.

6. In questo caso non esiste un caso peggiore e un caso migliore.

ALGO5(A)	
1	$a=0$
2	for $i=2$ to $A.length-1$
3	for $j=i-1$ to $i+1$
4	$a==a+A[j]$
5	return a

Per determinare la complessità asintotica del metodo osserviamo che l'istruzione alla riga 4 è l'istruzione dominante e quindi ci basta determinare quante volte essa viene eseguita. Il ciclo **for** più esterno viene ripetuto $\Theta(n)$ volte (per i che va da 2 a $n - 1$). Per ogni valore di i il ciclo più interno viene eseguito 3 volte (per $j = i - 1, i, i + 1$). Il numero di volte che l'istruzione dominante viene eseguita è quindi $\Theta(n)$, che è anche la complessità dell'algoritmo.

7. Il caso peggiore si ha quando nell'if delle righe 6-9 si esegue sempre il ramo **else**, cioè quando tutti gli elementi della matrice sono dispari. Il caso migliore si ha invece quando si esegue sempre il ramo **if**, cioè quando tutti gli elementi della matrice sono pari.

ALGO6(M)	
1	$c=0$
2	for $i=1$ to $M.rows$
3	$j=1$
4	while $j \leq M.columns$
5	$c=c+M[i][j]$
6	if $M[i][j] \bmod 2 == 0$
7	$j=j*2$
8	else
9	$j=j+1$
10	return c

Per determinare la complessità asintotica del metodo osserviamo che le istruzioni alle righe 5 e 6 sono le istruzioni dominanti e quindi ci basta determinare quante volte esse vengono eseguite. Il ciclo for più esterno viene ripetuto $\Theta(m)$ volte (per i che va da 1 a m). Il numero di ripetizioni del secondo ciclo variano in base al fatto che si esegua il ramo if o il ramo else. Nel caso peggiore j aumenta di una unità ad ogni iterazione e quindi il numero di ripetizioni del secondo ciclo è $\Theta(n)$ per ogni iterazione del primo. Nel caso peggiore quindi $T(m, n) = \Theta(m \cdot n)$. Nel caso migliore j raddoppia ad ogni iterazione. Procedendo come nel caso degli esercizi 3 e 4 si ottiene si può vedere che in questo caso il numero di ripetizioni del secondo ciclo è $\Theta(\lg n)$. Ne segue che nel caso migliore si ha $T(m, n) = \Theta(m \cdot \lg n)$.

8. In questo caso non esiste un caso peggiore e un caso migliore.

ALGO8(A)	
1	$k=0$
2	for $i=1$ to $A.length$
3	$k=k+i$
4	$a=0$
5	for $i=1$ to k
6	$j=i \bmod A.length$
7	$a=a+A[j]$
8	return a

Per determinare la complessità asintotica del metodo osserviamo che la complessità del primo ciclo (righe 2-3) è, asintoticamente parlando, pari al numero di volte che esso si ripete (infatti il suo corpo ha costo costante); quindi il primo ciclo ha complessità $\Theta(n)$. Analogamente il secondo ciclo ha complessità $\Theta(k)$. Per determinare la complessità dell'intero algoritmo è necessario capire quanto vale k in funzione di n . Il valore di k viene calcolato dal primo ciclo, ed è pari a $\sum_{i=1}^n i = \Theta(n^2)$. In definitiva il primo ciclo ha complessità $\Theta(n)$ mentre il secondo ha complessità $\Theta(n^2)$. L'intero algoritmo ha quindi complessità $\Theta(n^2)$.

9. In questo caso non esiste un caso peggiore e un caso migliore.

ALGO7(A)	
1	$a=0$
2	for $i=1$ to $A.length$
3	$j=1$
4	$k=1$
5	$continua=TRUE$
6	while $continua$
7	$a=A[i][j]$
8	if $j==A.length$ and $k<A.length$
9	$k=k+1$
10	$j=1$
11	elseif $j==A.length$ and $k==A.length$
12	$continua=FALSE$
13	else
14	$j=j+1$
15	return a

Per determinare la complessità asintotica del metodo osserviamo che le istruzioni alle righe 7 e 8 sono le istruzioni dominanti e quindi ci basta determinare quante volte esse vengono eseguite. Il ciclo `for` più esterno viene ripetuto $\Theta(n)$ volte (per i che va da 1 a n). Per capire quante volte viene ripetuto il ciclo interno dobbiamo capire come variano j e k . La variabile i viene inizializzata ad 1. Nelle prime $n - 1$ iterazioni del ciclo verrà eseguita l'istruzione alla riga 14, cioè j aumenta di 1 ad ogni iterazione (nel frattempo k non viene modificata). All'iterazione n -esima vengono eseguite le istruzioni alle righe 9 e 10 e quindi j viene riportata al valore 1 mentre k viene incrementata di 1. A questo punto ci sono altre $n - 1$ iterazioni in cui si incrementa soltanto j , seguite da una nuova esecuzione delle righe 9 e 10 che riassegnano 1 a j e incrementano k . In definitiva il secondo ciclo consiste di un certo numero di ripetizioni; in ogni ripetizione j assume tutti i valori da 1 a n . Ad ogni ripetizione il valore di k viene incrementato di 1. Il numero di ripetizioni è n , infatti quando sia j che k assumono il valore n , il ciclo termina. In definitiva il numero di iterazioni del secondo ciclo (per ogni iterazione del primo) è dato da $\sum_{k=1}^n n = n^2$. Ne segue che il numero di volte che viene ripetuta l'istruzione dominante è n^3 e quindi la complessità dell'algoritmo è $\Theta(n^3)$.